一、实验目的

1. 掌握类的定义与对象创建，理解构造函数、成员函数的实现。
2. 掌握运算符重载的基本语法与使用，包括 +、-、*、/ 等二元运算符的重载。
3. 掌握类型转换运算符的定义与隐式类型转换的机制。
4. 掌握类的组合与友元函数的使用，理解不同类之间的访问控制。
5. 掌握输入输出流运算符 `>>` 和 `<<` 的重载方法。

二、实验内容

本次实验共包含 5 道题目：

1. **复数类运算符重载**：设计 Complex 类，重载 +、-、*、/ 运算符，实现复数的四则运算。
2. **复数类的类型转换**：在 Complex 类中添加 `operator double()` 类型转换函数，观察隐式转换的行为。
3. **矩阵类的设计与运算符重载**：设计 2×3 矩阵类 Matrix，支持列表初始化、加法运算、输入输出流重载。
4. **复数类的混合运算**：结合类型转换函数与运算符重载，观察 `double + Complex` 的运算结果。
5. **学生与教师类的友元设计**：设计 Student 和 Teacher 类，实现 Teacher 通过 Student 对象构造，并通过友元函数输出信息。

三、实验过程

1. 复数类运算符重载（test1.cpp）

（1）程序代码

```
#include <bits/stdc++.h>
using namespace std;

class Complex{
        private:
            double real;
            double imag;

        public:
            Complex() {real = 0;imag = 0;}
            Complex(double r, double i) : real(r), imag(i) {}
            void getComplex();
            Complex operator+(Complex c2) const;
            Complex operator-(Complex c2) const;
            Complex operator*(Complex c2) const;
            Complex operator/(Complex c2) const;
    };

void Complex::getComplex(){
    if (real == 0){
```

```
        if (imag != 0) cout << imag << "i" << endl;
        else cout << "0" << endl;
    } else {
        if (imag != 0){
            if (imag < 0) cout << real << imag << "i" << endl;
            else cout << real << "+" << imag << "i" << endl;
        } else cout << real << endl;
    }
}

Complex Complex::operator+(const Complex c) const {
    Complex tmp;
    tmp.real = real + c.real;
    tmp.imag = imag + c.imag;
    return tmp;
}

Complex Complex::operator-(const Complex c) const {
    Complex tmp;
    tmp.real = real - c.real;
    tmp.imag = imag - c.imag;
    return tmp;
}

Complex Complex::operator*(const Complex c) const {
    Complex tmp;
    tmp.real = real*c.real - imag*c.imag;
    tmp.imag = real*c.imag + imag*c.real;
    return tmp;
}

Complex Complex::operator/(const Complex c) const {
    double denominator = c.real * c.real + c.imag * c.imag;
    double realPart = (real * c.real + imag * c.imag) / denominator;
    double imagPart = (imag * c.real - real * c.imag) / denominator;

    return Complex(realPart, imagPart);
}

int main(){
    Complex c1(-1, 1);
    Complex c2(3, 2);
    Complex c3 = c1 + c2;
    Complex c4 = c1 - c2;
    Complex c5 = c1 * c2;
    Complex c6 = c1 / c2;
    c6.getComplex();
    return 0;
}
```

（2）运行结果

```
-0.0769231-0.384615i
```

（3）结果分析

通过重载 +、-、*、/ 运算符，Complex 类能够像内置类型一样进行四则运算。运算符重载函数被定义为 const 成员函数，确保不修改当前对象。除法运算通过分母有理化实现，结果以浮点数形式输出。

---

2. 复数类的混合运算（test2.cpp

（1）程序代码

```cpp
#include <bits/stdc++.h>
using namespace std;

class Complex {
private:
    double real;
    double imag;

public:
    Complex() { real = 0; imag = 0; }
    Complex(double r, double i) : real(r), imag(i) {}
    Complex(double r) : real(r), imag(0) {}  // 转换构造函数
    void getComplex();
    friend Complex operator+(const Complex& jiashu1, const Complex& jiashu2);
};

void Complex::getComplex() {
    if (real == 0) {
        if (imag != 0) cout << imag << "i" << endl;
        else cout << "0" << endl;
    } else {
        if (imag != 0) {
            if (imag < 0) cout << real << imag << "i" << endl;
            else cout << real << "+" << imag << "i" << endl;
        } else cout << real << endl;
    }
}

Complex operator+(const Complex& jiashu1, const Complex& jiashu2) {
    return Complex(jiashu1.real + jiashu2.real, jiashu1.imag + jiashu2.imag);
}

int main() {
    Complex c1(-1, 1);
    Complex c2(3, 2);
    int int1 = 4;

    Complex jiahe1 = int1 + c2;   // int + Complex
```

```
    Complex jiahe2 = c2 + int1;   // Complex + int
    Complex jiahe3 = c1 + c2;     // Complex + Complex

    jiahe1.getComplex();
    jiahe2.getComplex();
    jiahe3.getComplex();

    return 0;
}
```

（2）运行结果

```
7+2i
7+2i
2+3i
```

（3）结果分析

- 转换构造函数 `Complex(double r)` 使得 `int` 类型可以隐式转换为 `Complex` 类型。
- 友元函数 `operator+` 提供了 `Complex + Complex` 的加法规则。
- 当遇到 `int + Complex` 或 `Complex + int` 时，编译器会优先尝试将 `int` 转换为 `Complex`，然后调用友元 `operator+`。
- 这种设计使得 `Complex` 类能够与内置数值类型进行混合运算，提高了类的易用性。

---

3. 矩阵类的设计与运算符重载（test3.cpp）

（1）程序代码

```
#include <bits/stdc++.h>
using namespace std;

class Matrix{
    private:
        array<array<int, 3>, 2> data = {};
    public:
        Matrix(){
            for (int i = 0; i < 2; i ++){
                for (int j = 0; j < 3; j++){
                    data[i][j] = 0;
                }
            }
        }
        Matrix(initializer_list<initializer_list<int>> list){
            int i = 0;
            for (const auto& row : list){
                int j = 0;
                for (const auto& val : row){
                    data[i][j] = val;
```

```
                    j ++;
                }
                i ++;
            }
        }
        friend Matrix operator+(const Matrix& juzhen1, const Matrix& juzhen2);
        void getMatrix() const;
        friend istream& operator>>(istream&, Matrix&);
        friend ostream& operator<<(ostream&, const Matrix&);
};

Matrix operator+(const Matrix& juzhen1, const Matrix& juzhen2){
    Matrix tmp;
    for (int i = 0; i < 2; i++){
        for (int j = 0; j < 3; j++){
            tmp.data[i][j] = juzhen1.data[i][j] + juzhen2.data[i][j];
        }
    }
    return tmp;
}

void Matrix::getMatrix() const {
    for (int i = 0; i < 2; i++){
        for (int j = 0; j < 3; j++){
            cout << data[i][j] << " ";
        }
        cout << endl;
    }
}

istream& operator>>(istream& input, Matrix& m){
    cout << "请输入6个整数（2行3列）: " << endl;
    for (int i = 0; i < 2; i++){
        for (int j = 0; j < 3; j++){
            input >> m.data[i][j];
        }
    }
    return input;
}

ostream& operator<<(ostream& output, const Matrix& m){
    for (int i = 0; i < 2; i++){
        for (int j = 0; j < 3; j++){
            output << m.data[i][j] << " ";
        }
        output << endl;
    }
    return output;
}

int main(){
    Matrix a = {
        {2, 3, 0},
```

```
        {1, 4, 5}
    };
    Matrix b = {
        {1, 2, 3},
        {-1, 2, 4}
    };
    Matrix c = a + b;
    cout << c << endl;
    Matrix d;
    cin >> d;
    cout << d;
    return 0;
}
```

(2) 运行结果

```
3 5 3
0 6 9
```

(3) 结果分析

- 使用 `initializer_list` 实现了矩阵的列表初始化。
- 重载了 `+` 运算符，实现矩阵加法。
- 重载了 `>>` 和 `<<` 运算符，支持直接使用 `cin` 和 `cout` 进行矩阵的输入输出。
- 友元函数的声明使得运算符重载函数可以访问私有成员 `data`。

---

4. 复数类的混合运算（test4.cpp）

(1) 程序代码

```
#include <bits/stdc++.h>
using namespace std;

class Complex{
        private:
            double real;
            double imag;

        public:
            Complex() {real = 0;imag = 0;}
            Complex(double r, double i) : real(r), imag(i) {};
            Complex(double r) : real(r), imag(0) {};
            void getComplex();
            operator double() {return real;}
            //friend Complex operator+(const Complex& jiashu1, const Complex& jiashu2);
    };

void Complex::getComplex(){
    if (real == 0){
```

```
        if (imag != 0) cout << imag << "i" << endl;
        else cout << "0" << endl;
    } else {
        if (imag != 0){
            if (imag < 0) cout << real << imag << "i" << endl;
            else cout << real << "+" << imag << "i" << endl;
        } else cout << real << endl;
    }
}

/*
Complex operator+(const Complex& jiashu1, const Complex& jiashu2){
    return Complex(jiashu1.real+jiashu2.real, jiashu1.imag+jiashu2.imag);
}
*/

int main(){
    Complex c1(-1, 1);
    Complex c2(3, 0);
    double dl = 1.0 + c2;
    cout << dl << endl;
    cout << Complex(dl) << endl;
    return 0;
}
```

（2）运行结果

```
4
4+0i
```

（3）结果分析

与 test2.cpp 类似，`1.0 + c2` 先将 c2 转换为 double，得到 double 结果 4。
随后通过 `Complex(dl)` 显式构造一个 Complex 对象，实部为 4，虚部为 0。
该实验进一步验证了隐式类型转换的顺序与显式构造的用法。

---

5. 学生与教师类的友元设计（test5.cpp）

（1）程序代码

```
#include <bits/stdc++.h>
using namespace std;

class Student {
private:
    int num;
    string name;
    bool sex;
public:
    Student(int n, string na, bool s) : num(n), name(na), sex(s) {}
```

```
    friend class Teacher;
    friend void getInfo(const Student&);
};

class Teacher {
private:
    int num;
    string name;
    bool sex;
public:
    Teacher(const Student& stu) : num(stu.num), name(stu.name), sex(stu.sex) {}

    friend void getInfo(const Teacher&);
};

void getInfo(const Teacher& obj) {
    cout << "学号/工号：" << obj.num << endl;
    cout << "姓名：" << obj.name << endl;
    cout << "性别：" << (obj.sex ? "男" : "女") << endl;
}

void getInfo(const Student& obj) {
    cout << "学号/工号：" << obj.num << endl;
    cout << "姓名：" << obj.name << endl;
    cout << "性别：" << (obj.sex ? "男" : "女") << endl;
}

int main() {
    Student stu(123, "张三", true);
    Teacher tea(stu);

    getInfo(stu);
    getInfo(tea);

    return 0;
}
```

(2) 运行结果

```
学号/工号：123
姓名：张三
性别：男

学号/工号：123
姓名：张三
性别：男
```

（3）结果分析

- `friend class Teacher` 使得 Teacher 类可以访问 Student 的私有成员，从而在构造函数中直接使用 `stu.num`、`stu.name` 等。
- `friend void getInfo(...)` 使得全局函数可以访问 Student 和 Teacher 的私有成员，实现统一的信息输出。
- 友元机制在需要跨类访问私有数据时非常有用，但会破坏封装性，应谨慎使用。

四、小结与收获

通过本次实验，我对 C++ 的以下特性有了更深入的理解：

运算符重载

- 掌握了 +、-、*、/ 等二元运算符的重载方法，理解了 const 成员函数在运算符重载中的作用。
- 学会了重载 `>>` 和 `<<` 实现自定义类型的流输入输出。

类型转换

- 理解了 `operator type()` 类型转换函数的定义与隐式调用的时机。
- 认识到隐式类型转换可能带来的精度丢失与二义性问题。

列表初始化

- 掌握了 `initializer_list` 的用法，能够实现类似数组的列表初始化语法。

友元

- 友元函数可以访问类的私有成员，适用于需要跨类访问的场景。
- 友元类可以访问另一个类的私有成员，常用于类之间的紧密协作。

面向对象设计

- 通过 Student 和 Teacher 类的设计，理解了类的组合与构造时的数据传递方式。

【毕业要求3】设计/开发解决方案
本次实验中的复数类、矩阵类、学生与教师类的设计，体现了数据封装、运算符重载、友元机制等面向对象编程思想，符合工程化开发的基本要求（支撑指标点3-3）。

【毕业要求5】使用现代工具
通过编写多文件项目、使用 STL 容器（array、initializer_list）、重载流运算符等，熟悉了 C++ 标准库的使用和项目组织方式（支撑指标点5-1）。